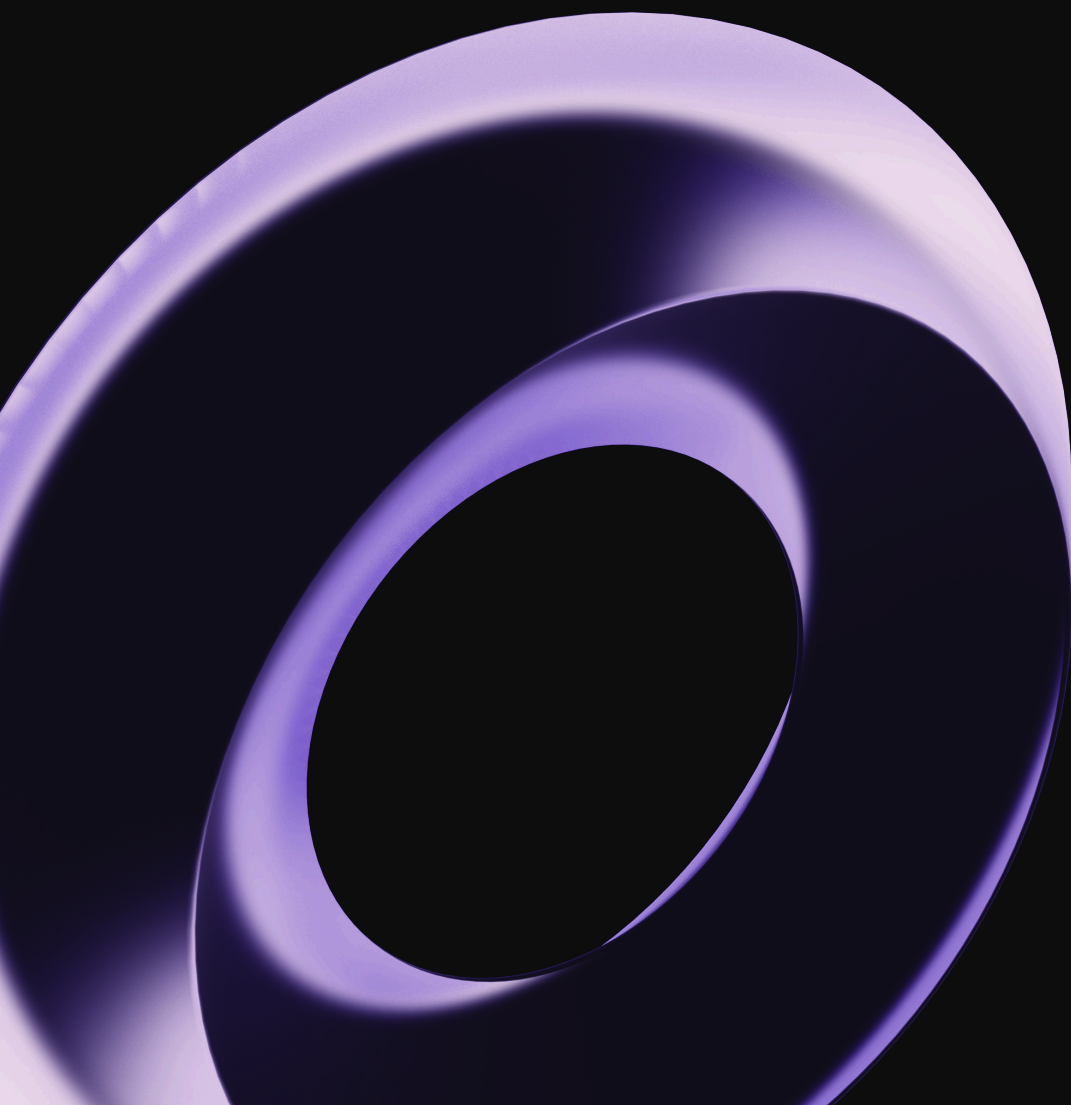


Microservices Architecture Essentials

Agenda

1. Introduction and Building Microservices
2. Microservices Architecture Patterns and Communication
3. Advanced Microservices and Operational Practices





Introduction and Building Microservices

Course Agenda



Core Principles of Microservices

Decouple services for flexibility and independent scaling.

Defining Service Boundaries

Use Domain-Driven Design to clarify responsibilities.

Transitioning from Monoliths

Apply the Strangler Fig pattern for gradual migration.

Building Cloud-Native Apps

Leverage the 12-Factor App methodology for scalability.



Introduction & Building Standalone Microservices

Understanding Microservices

- Small, independent services focused on specific business functions.
- Contrast with monolithic and SOA architectures for flexibility.

Building Standalone Services

- Create your first microservice using Express for simplicity.
- Develop scalable services with NestJS and integrate TypeORM & MongoDB.

Project Structure & Best Practices

- Organize code for maintainability and follow coding standards.

Docker Basics for Microservices

- Containerize services for consistent deployment and scalability.



Docker Basics for Microservices

Containerize Microservices with Docker

Package each service and its dependencies into isolated containers.

Simplified Deployment and Scalability

Deploy services consistently across environments.

Improved Resource Efficiency

Run multiple containers on a single host efficiently.

Faster Development and Testing

Quickly spin up, update, or roll back services as needed.



Microservices Architecture Patterns and Communication

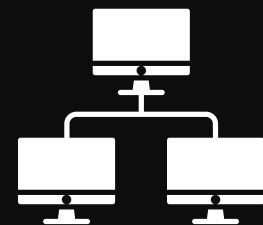


Module 2: Microservices Architecture Patterns (with Code Demos)



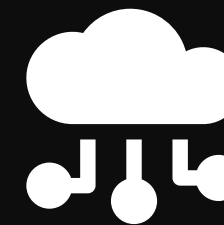
Service Communication Patterns

- REST, gRPC, and message queues for flexible interactions
- API Gateway centralizes access and security
- Service discovery enables dynamic connectivity



Database and Event Patterns

- Database per service ensures autonomy
- Avoid shared database anti-patterns
- Event-driven architecture for scalability



Advanced Distributed Patterns

- Saga pattern manages distributed transactions
- CQRS and event sourcing for robust data handling

Communication Patterns



REST: Simplicity and Ubiquity

- Ideal for stateless, HTTP-based communication.
- Widely supported and easy to implement.

gRPC: High Performance RPC

- Efficient binary protocol for low-latency needs.
- Best for internal service-to-service calls.

Message Queues: Asynchronous Messaging

- Enables decoupled, event-driven workflows.
- Useful for scaling and handling spikes in traffic.

Choosing the Right Pattern

- REST for public APIs, gRPC for internal, queues for async tasks.

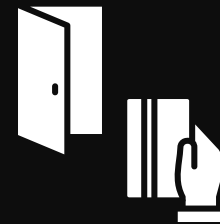


API Gateway and Service Discovery



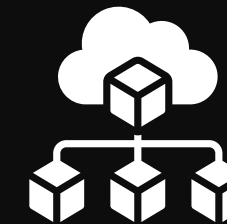
Centralized API Gateway

- Acts as a single entry point for all client requests
- Enables unified authentication, rate limiting, and logging



Unified Security and Access Control

- Simplifies enforcement of security policies across services



Service Discovery Mechanisms

- Allows services to dynamically locate and communicate with each other
- Supports scaling and resilience in distributed systems



Module 3: Architecture Patterns & Real Use Cases

E-commerce Platform Architecture

Microservices enable modular, scalable online stores.

Real-Time Notification Systems

Event-driven services deliver instant user updates.

Payment Processing Pipelines

Isolated payment services ensure secure transactions.

Inventory & Order Management

Sagas coordinate inventory and order workflows.

Analytics & Reporting with CQRS

CQRS separates read/write for efficient analytics.



Advanced Microservices and Operational Practices



Module 4: NestJS Native Microservices



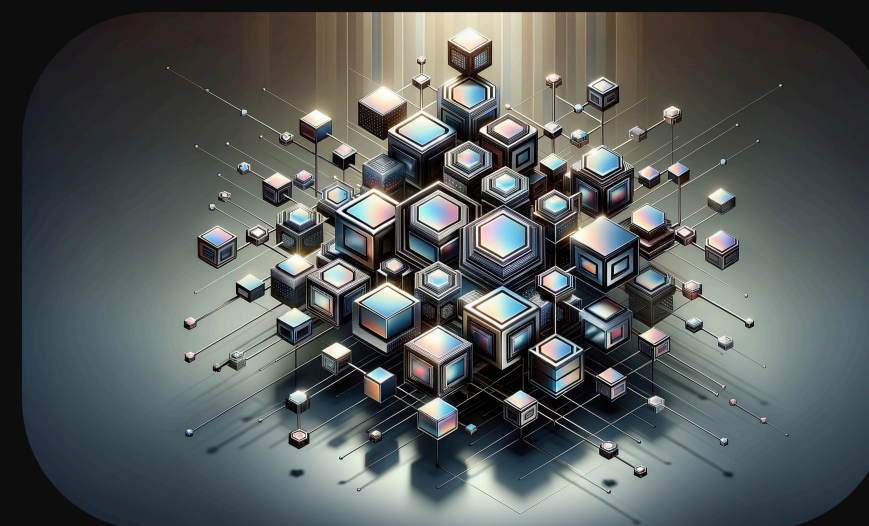
Diverse Transport Options in NestJS

Support for TCP, AMQP/RabbitMQ, gRPC, and Redis Pub/Sub transports



Hybrid Microservice Applications

Combine multiple transport layers for flexible architectures



Custom Transporters

Extend NestJS with custom protocols for unique needs



Bonus Modules

Serverless Microservices with AWS Lambda

Leverage AWS Lambda for scalable, event-driven services.

API Gateway Solutions

Integrate Kong or AWS API Gateway for unified API management.

CI/CD Pipelines for Microservices

Automate build, test, and deployment for rapid delivery.

Monitoring and Distributed Tracing

Use Prometheus, Grafana, and ELK stack for observability.



Key Takeaways and Next Steps

Core Principles of Microservices

Decentralized, independently deployable services

Key Architectural Patterns

REST, gRPC, message queues, API Gateway, event-driven

Resilience and Scalability

Embrace patterns like circuit breaker, saga, and CQRS

Continuous Learning and Practice

Experiment with real-world scenarios and hands-on projects